

Compilation

Sélection d'instruction et mise en pile

Adrien Guatto

Master 1 Informatique

Semestre 1 – 2025

Plan

1 Introduction

2 Structure de la passe

3 Conclusion

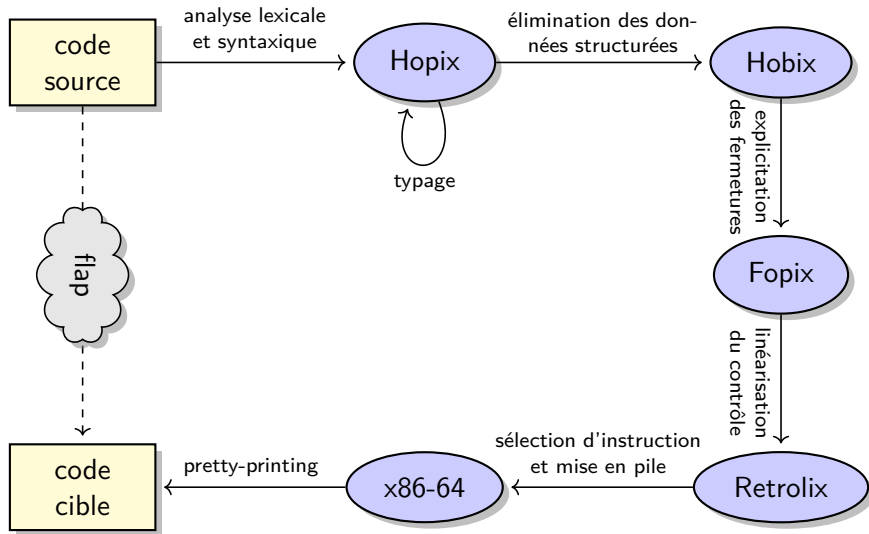
Plan

1 Introduction

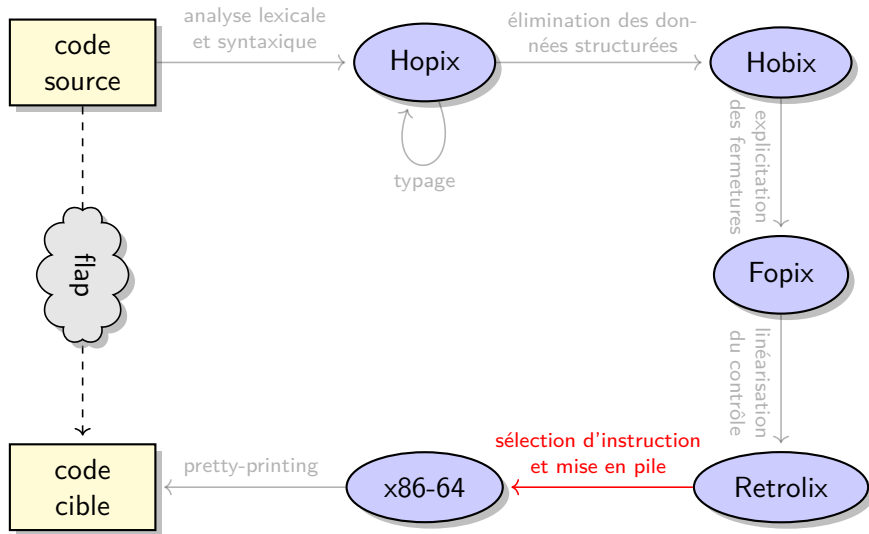
2 Structure de la passe

3 Conclusion

Le flot de compilation de flap



Le flot de compilation de flap



Le langage Retrolix (1/2)

```
type program = definition list
```

```
and definition =
```

```
| DValues    of identifier list * block
```

```
| DFunction  of function_identifier * identifier list * block
```

```
| DExternalFunction of function_identifier
```

```
and block = identifier list * labelled_instruction list
```

```
and labelled_instruction = label * instruction
```

Le langage Retrolix (2/2)

and instruction =

- | **Call of** rvalue * rvalue **list** * **bool**
- | **Ret**
- | **Assign of** lvalue * op * rvalue **list**
- | **Jump of** label
- | **ConditionalJump of** condition * rvalue **list** * label * label
- | **Switch of** rvalue * label **array** * label option
- | **Comment of** **string**
- | **Exit**

and op = **Copy** | **Add** | **Mul** | **Div** | **Sub** | **And** | **Or**

and condition = **GT** | **LT** | **GTE** | **LTE** | **EQ**

and rvalue = [lvalue | **`Immediate of** literal]

and lvalue = [**`Variable of** identifier | **`Register of** register]

and register = **RId of** **string**

Le langage x86-64 (extrait)

```
type reg = X86_64_Architecture.register
type label = string
type lit = Mint.t
type scale = [ `One | `Two | `Four | `Eight ]
type imm = Lit of lit | Lab of label
type address = { offset : imm option; base : reg option;
                 idx : reg option; scale : scale; }
type dst = [ `Addr of address | `Reg of reg ]
type src = [ dst | `Imm of imm ]
type suffix = [ `b | `w | `l | `q ]
type concode = E | ... | BE
type instruction =
  | Add of { s : suffix; src : src; dst : dst; }
  ...
  | Cmov of { cc : concode; s : suffix; src : reg; dst : dst; }
type directive = Section of string | ...
type line = Directive of directive | Label of label
           | Instruction of instruction
type t = line list
```


De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).
- Les instructions Retrolix sont moins nombreuses et plus structurées.

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).
- Les instructions Retrolix sont moins nombreuses et plus structurées.
 - Code dit *trois "adresses"* `dst <- op(src1, src2)` à la RISC.

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).
- Les instructions Retrolix sont moins nombreuses et plus structurées.
 - Code dit *trois "adresses"* `dst <- op(src1, src2)` à la RISC.
- Retrolix dispose d'une notion de variable locale primitive.

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).
- Les instructions Retrolix sont moins nombreuses et plus structurées.
 - Code dit *trois "adresses"* `dst <- op(src1, src2)` à la RISC.
- Retrolix dispose d'une notion de variable locale primitive.
- Retrolix dispose d'une notion de fonction, qui plus est :

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).
- Les instructions Retrolix sont moins nombreuses et plus structurées.
 - Code dit *trois "adresses"* `dst <- op(src1, src2)` à la RISC.
- Retrolix dispose d'une notion de variable locale primitive.
- Retrolix dispose d'une notion de fonction, qui plus est :
 - avec des paramètres explicites,

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).
- Les instructions Retrolix sont moins nombreuses et plus structurées.
 - Code dit *trois "adresses"* `dst <- op(src1, src2)` à la RISC.
- Retrolix dispose d'une notion de variable locale primitive.
- Retrolix dispose d'une notion de fonction, qui plus est :
 - avec des paramètres explicites,
 - avec appels possiblement terminaux (*tail*).

De Retrolix à x86-64

Question

Quelles sont les similarités et différences entre Retrolix et x86-64 ?

- Dans les deux cas, un programme est une séquence d'instructions élémentaires ; le contrôle est implémenté par des sauts (GOTO).
- Les instructions Retrolix sont moins nombreuses et plus structurées.
 - Code dit *trois "adresses"* `dst <- op(src1, src2)` à la RISC.
- Retrolix dispose d'une notion de variable locale primitive.
- Retrolix dispose d'une notion de fonction, qui plus est :
 - avec des paramètres explicites,
 - avec appels possiblement terminaux (*tail*).
- Un programme assembleur x86-64 doit comprendre des détails "protocolaires" nécessaires à l'insertion dans le reste du système.

Le module `RetrolixToX86_64`

Résumé de la tâche de la passe

- Passer d'un langage bas niveau abstrait au langage d'assemblage.
- Implémenter les instructions abstraites par des instructions machines.
- Éliminer les variables locales et paramètres via l'utilisation de la pile.

Le module `RetrolixToX86_64`

Résumé de la tâche de la passe

- Passer d'un langage bas niveau abstrait au langage d'assemblage.
- Implémenter les instructions abstraites par des instructions machines.
- Éliminer les variables locales et paramètres via l'utilisation de la pile.

Le code fourni gère tous les détails “protocolaires”, mais nécessite :

- l'emplacement mémoire de chaque variable locale et paramètre,
- une traduction x86-64 pour chaque instruction Retrolix.

Le module `RetrolixToX86_64`

Résumé de la tâche de la passe

- Passer d'un langage bas niveau abstrait au langage d'assemblage.
- Implémenter les instructions abstraites par des instructions machines.
- Éliminer les variables locales et paramètres via l'utilisation de la pile.

Le code fourni gère tous les détails “protocolaires”, mais nécessite :

- l'emplacement mémoire de chaque variable locale et paramètre,
- une traduction x86-64 pour chaque instruction Retrolix.

La sélection d'instruction et mise en pile

Le module `RetrolixToX86_64`

Résumé de la tâche de la passe

- Passer d'un langage bas niveau abstrait au langage d'assemblage.
- Implémenter les instructions abstraites par des instructions machines.
- Éliminer les variables locales et paramètres via l'utilisation de la pile.

Le code fourni gère tous les détails “protocolaires”, mais nécessite :

- l'emplacement mémoire de chaque variable locale et paramètre,
- une traduction x86-64 pour chaque instruction Retrolix.

La **sélection d'instruction** et mise en pile

- 1 insn. arithmétique/logique Retrolix $\rightarrow$ 1/2/... insn. x86-64
 - De petites difficultés (restriction 1 opérande mémoire max en x86-64).
 - Plusieurs choix possibles, plus ou moins performants.

Le module `RetrolixToX86_64`

Résumé de la tâche de la passe

- Passer d'un langage bas niveau abstrait au langage d'assemblage.
- Implémenter les instructions abstraites par des instructions machines.
- Éliminer les variables locales et paramètres via l'utilisation de la pile.

Le code fourni gère tous les détails “protocolaires”, mais nécessite :

- l'emplacement mémoire de chaque variable locale et paramètre,
- une traduction x86-64 pour chaque instruction Retrolix.

La sélection d'instruction et mise en pile

- 1 insn. arithmétique/logique Retrolix $\rightarrow$ 1/2/... insn. x86-64
 - De petites difficultés (restriction 1 opérande mémoire max en x86-64).
 - Plusieurs choix possibles, plus ou moins performants.
- Gérer la pile (prologue/épilogue/appels), calculer les emplacements.

Plan

1 Introduction

2 Structure de la passe

3 Conclusion

Modules à implémenter

Sélection d'instruction

```
module type InstructionSelector = sig
  (** [mov ~dst ~src] generates the x86-64 assembly listing to
      copy [src] into [dst]. *)
  val mov : dst:T.dst -> src:T.src -> T.line list
  (** [add ~dst ~src1 ~srcr] generates the x86-64 assembly
      listing to store [src1 + srcr] into [dst]. *)
  val add : dst:T.dst -> src1:T.src -> srcr:T.src -> T.line list
  ...
  (** [conditional_jump ~cc ~src1 ~srcr ~l1 ~lr] generates the
      x86-64 assembly listing to test whether [src1, srcr]
      satisfies the relation described by [cc] and jump to [l1]
      if they do or to [lr] when they do not. *)
  val conditional_jump :
    cc:T.condcode -> src1:T.src -> srcr:T.src ->
    l1:T.label -> lr:T.label -> T.line list
end
```


Modules à implémenter

Mise en pile (1/2)

```
module type FrameManager = sig
  (** The abstract data structure holding the information
      necessary to implement the calling convention. *)
  type frame_descriptor
  (** Generate a frame descriptor for the function with
      parameter [params] and locals [locals]. *)
  val frame_descriptor : params:S.identifier list
    -> locals:S.identifier list -> frame_descriptor
  (** [location_of fd v] computes the address of [v] according
      to the frame descriptor [fd]. Note that [v] might be a
      local variable, a function parameter, or a global variable. *)
  val location_of : frame_descriptor -> S.identifier -> T.address
  ...
end
```

Modules à implémenter

Mise en pile (2/2)

```
...
(** [function_prologue fd] generates the x86-64 assembly
    listing to setup a stack frame according to the frame
    descriptor [fd]. *)
val function_prologue : frame_descriptor -> T.line list
(** [function_epilogue fd] generates the x86-64 assembly
    listing to setup a stack frame according to the frame
    descriptor [fd]. *)
val function_epilogue : frame_descriptor -> T.line list
(** [call fd ~kind ~f ~args] generates the x86-64 assembly
    listing to setup a call to the function at [f], with
    arguments [args], with [kind] specifying whether this
    should be a normal or tail call. *)
val call :
  frame_descriptor -> kind:[ `Normal | `Tail ] ->
  f:T.src -> args:T.src list -> T.line list
end
```

Quid des appels terminaux ?

Définition

Un appel est *terminal* s'il n'a pas de continuation dans l'appelant.

Pourquoi ?

Nécessaire à la compilation **correcte** des fonctions récursives terminales.

Quid des appels terminaux ?

Définition

Un appel est *terminal* s'il n'a pas de continuation dans l'appelant.

Pourquoi ?

Nécessaire à la compilation correcte des fonctions récursives terminales.

- Le retour de l'appel doit donc se faire...dans l'appelant de l'appelant !
 - Sauf si le grand-père avait lui-même fait un appel terminal, etc.
- Comment l'implémenter ?

Quid des appels terminaux ?

Définition

Un appel est *terminal* s'il n'a pas de continuation dans l'appelant.

Pourquoi ?

Nécessaire à la compilation correcte des fonctions récursives terminales.

- Le retour de l'appel doit donc se faire...dans l'appelant de l'appelant !
 - Sauf si le grand-père avait lui-même fait un appel terminal, etc.
- Comment l'implémenter ? Libérer son cadre de pile avant l'appel.
 - L'adresse de retour sur la pile dans l'appelé sera donc, par définition, celle du dernier appel non-terminal, et RET fonctionnera.

Quid des appels terminaux ?

Définition

Un appel est *terminal* s'il n'a pas de continuation dans l'appelant.

Pourquoi ?

Nécessaire à la compilation correcte des fonctions récursives terminales.

- Le retour de l'appel doit donc se faire...dans l'appelant de l'appelant !
 - Sauf si le grand-père avait lui-même fait un appel terminal, etc.
- Comment l'implémenter ? ~ Libérer son cadre de pile avant l'appel.
 - L'adresse de retour sur la pile dans l'appelé sera donc, par définition, celle du dernier appel non-terminal, et RET fonctionnera.

Plan

1 Introduction

2 Structure de la passe

3 Conclusion

Conclusion

La sélection d'instruction et mise en pile

- Nécessaires à tout compilateur qui produit du code machine.
- Très simple et naïf dans la version qu'on a présenté.
 - Manque à notre mise en pile une *allocation de registre* préalable.

Conclusion

La sélection d'instruction et mise en pile

- Nécessaires à tout compilateur qui produit du code machine.
- Très simple et naïf dans la version qu'on a présenté.
 - Manque à notre mise en pile une *allocation de registre* préalable.

Dans un compilateur optimisant, les deux tâches sont distinguées.

Conclusion

La sélection d'instruction et mise en pile

- Nécessaires à tout compilateur qui produit du code machine.
- Très simple et naïf dans la version qu'on a présenté.
 - Manque à notre mise en pile une *allocation de registre* préalable.

Dans un compilateur optimisant, les deux tâches sont distinguées.

Sélection d'instruction

- Algorithmes globaux plutôt que par instruction ; simplifications.
 - Particulièrement pertinent pour cibles CISC comme x86-64.
- Bien compris dans sa version classique, de nouveaux sujets.

Conclusion

La sélection d'instruction et mise en pile

- Nécessaires à tout compilateur qui produit du code machine.
- Très simple et naïf dans la version qu'on a présenté.
 - Manque à notre mise en pile une *allocation de registre* préalable.

Dans un compilateur optimisant, les deux tâches sont distinguées.

Sélection d'instruction

- Algorithmes globaux plutôt que par instruction ; simplifications.
 - Particulièrement pertinent pour cibles CISC comme x86-64.
- Bien compris dans sa version classique, de nouveaux sujets.

Allocation de registre

- Avant la mise en pile ; traduction Retrolix → Retrolix dans flap.
- Transforme les variables en registres. Bien plus rapide !

Annexes

Optimisations locales dans GCC

```
/* PR53979: Transform ((a ^ b) | a) -> (a | b) */
```

```
(simplify
```

```
  (bit_ior:c (bit_xor:c @0 @1) @0)
```

```
  (bit_ior @0 @1))
```

```
/* (a & ~b) | (a ^ b)  -->  a ^ b  */
```

```
(simplify
```

```
  (bit_ior:c (bit_and:c @0 (bit_not @1)) (bit_xor:c@2 @0 @1))  
  @2)
```

```
...
```

gcc/match.pd (12 kLoC)